



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Sistema de Visión Artificial
para el Mantenimiento de
Carreteras**



Presentado por Marcos Gómez Vega
en Universidad de Burgos — 8 de julio de 2025
Tutor: Bruno Baruque Zano



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D.Bruno Baruque Zanon, profesor del departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial.

Expone:

Que el alumno D. Marcos Gómez Vega, con DNI 713099554D, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Sistema de Visión Artificial para el Mantenimiento de Carreteras.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 8 de julio de 2025

Vº. Bº. del Tutor:

D. Bruno Baruque Zanon

Resumen

El TFG titulado *Sistema de Visión Artificial para el Mantenimiento de Carreteras* consiste en una aplicación móvil para dispositivos Android, mediante la cual los usuarios pueden tomar fotografías de desperfectos en la vía, como grietas o baches. Posteriormente, un modelo de Inteligencia Artificial detecta automáticamente el tipo de incidencia. El usuario puede entonces enviar dicha incidencia a las autoridades, incluyendo la imagen, el tipo de desperfecto y la geolocalización, con el fin de facilitar su reparación.

Descriptores

Visión artificial, mantenimiento de la vía, inteligencia artificial, Android, detección de incidencias, geolocalización, aplicación móvil, reconocimiento de imágenes.

Abstract

The Bachelor's Thesis entitled *Artificial Vision System for Road Maintenance* consists of a mobile application for Android devices that allows users to take photographs of road damage, such as cracks or potholes. Subsequently, an Artificial Intelligence model automatically detects the type of issue. The user can then send the incident to the authorities, including the image, the type of damage, and the geolocation, in order to facilitate its repair.

Keywords

Artificial vision, track maintenance, artificial intelligence, Android, incident detection, geolocation, mobile application, image recognition.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. App detección de incidencias	1
1.2. Estructura del Trabajo	2
1.3. Materiales Complementarios	3
2. Objetivos del proyecto	4
2.1. Objetivos generales	4
2.2. Objetivos técnicos	5
3. Conceptos teóricos	6
3.1. Inteligencia Artificial y Aprendizaje Automático	6
3.2. Aprendizaje Profundo (Deep Learning)	6
3.3. Detección de Objetos y Box Bounding	7
3.4. Entrenamiento de Modelos de IA	7
3.5. Modelo MobileNetV2	7
4. Técnicas y herramientas	9
4.1. Android Studio	9
4.2. Servicios en la nube	10
4.3. Framework de Inteligencia Artificial	11
4.4. Modelo de IA usado	12

4.5. Dataset utilizados	14
4.6. Control de versiones	15
5. Aspectos relevantes del desarrollo del proyecto	16
5.1. Desarrollo e integración del modelo de clasificación con Mobi- leNetV2	16
5.2. Creación de la plataforma web para el usuario master	20
6. Trabajos relacionados	23
6.1. Aplicaciones Analizadas	23
6.2. Patrones UX Identificados	26
6.3. Comparativa	27
6.4. Vacíos y Oportunidades	27
6.5. Conclusión	28
7. Conclusiones e informe crítico	29
7.1. Conclusiones	29
7.2. Informe crítico y posibles mejoras	29
Bibliografía	31

Índice de figuras

5.1. Matriz de confusión del modelo	19
5.2. Web Administrador	21

Índice de tablas

6.1. Comparativa de aplicaciones para reporte de incidencias urbanas	27
--	----

1. Introducción

1.1. App detección de incidencias

Desde hace no muchos años, la inteligencia artificial está incorporándose en muchos aspectos de la vida, y es una realidad lo mucho que nos ayuda a realizar acciones habituales. Gracias a la inteligencia artificial, estas tareas son normalmente más rápidas y, en muchos casos, más fiables.

Antiguamente, para arreglar un desperfecto, las mismas organizaciones encargadas de repararlos tenían que ir a comprobar cómo era este desperfecto para, de ese modo, llevar la maquinaria especializada necesaria para poder arreglarlo.

Con la llegada de la tecnología y de los teléfonos móviles, que son una realidad completamente instaurada en la sociedad y de los que muchas personas no podrían prescindir, se crearon aplicaciones para detectar incidencias a través de una foto. Sin embargo, el usuario era el responsable de indicar qué tipo de incidencia era. Es decir, cuando el usuario registraba una incidencia, hacía una foto y elegía el tipo de incidencia, como por ejemplo, una grieta.

Estas aplicaciones tienen un gran problema, y es que el usuario podía equivocarse al seleccionar el tipo de incidencia, o no ser lo suficientemente preciso. Por ejemplo, en esas aplicaciones, cuando se veía una grieta en la carretera, no se podía elegir el tamaño de la grieta, ya que era algo abstracto.

Con la inteligencia artificial, lo que podemos hacer es reducir estos errores de detección del tipo de incidencia y hacer que sean más precisos. Los modelos de inteligencia artificial no siempre son los más fiables, pero para eso está el usuario, quien antes de enviar la incidencia, comprueba que el modelo haya identificado correctamente el tipo de incidencia.

También es muy importante, para la rapidez a la hora de arreglar las incidencias, geo-localizar dónde se encuentran. Para ello, desde la aplicación móvil, al momento de enviar la incidencia, se obtiene la localización de la foto a través del GPS. Esta información, junto con la foto y el tipo de incidencia, se envía a las autoridades para que puedan actuar más rápido y saber qué tipo de herramientas deben llevar.

Además, la integración de inteligencia artificial en aplicaciones móviles para la detección de incidencias no solo mejora la eficiencia en la resolución de problemas en infraestructuras, sino que también tiene un impacto significativo en la optimización de recursos y en la reducción de tiempos de respuesta. Este tipo de tecnologías tiene aplicaciones potenciales en sectores tan diversos como la seguridad vial, la construcción, y la gestión de infraestructuras públicas, donde la detección temprana de fallos puede evitar accidentes, reducir costos de mantenimiento y garantizar una mayor seguridad para los usuarios. La automatización en la detección de incidencias permite, además, una gestión más eficiente de las incidencias a través de la recopilación y análisis de grandes volúmenes de datos, lo que puede llevar a la mejora continua de los sistemas de predicción y de toma de decisiones.

1.2. Estructura del Trabajo

La memoria tiene la siguiente estructura:

- **Introducción:** Breve introducción donde se plantea un poco cuáles son las bases del proyecto, de dónde surge, y trata muy por encima qué objetivos básicos pretende conseguir y de qué manera.
- **Objetivos del proyecto:** En esta sección se pretende hablar más detenida y profundamente de los objetivos concretos del proyecto, haciendo un análisis más exhaustivo de los requisitos software que han de cumplirse.
- **Conceptos teóricos:** Aquí se explicarán todos los conceptos teóricos que se han utilizado en el desarrollo de este TFG, y que son necesarios conocer para comprender algunos aspectos importantes del proyecto.
- **Técnicas y herramientas:** Sección dirigida a tratar las diferentes técnicas de planificación y herramientas de trabajo utilizadas en este proyecto.

- **Aspectos relevantes del desarrollo del proyecto:** Aquí se va a detallar la evolución del proyecto, desde sus inicios hasta el resultado final, explicando detalladamente todas las fases del mismo.
- **Trabajos relacionados:** En esta sección se hablará sobre otros trabajos similares que hay sobre aplicaciones móviles que tectan incidencias, sean de pago, gratisitos y que tengan que ver con el tema del trabajo.
- **Conclusiones y líneas de trabajo futuras:** Sección para tratar las conclusiones de la realización del trabajo y para plantear cómo podría ser ampliado este proyecto.

1.3. Materiales Complementarios

- **Memoria.pdf:** Esta misma memoria en formato PDF.
- **Código fuente:** Incluye una copia del código fuente de la aplicación.
- **Modelo de IA entrenado:** El modelo entrenado de inteligencia artificial utilizado en el proyecto.
- **Documentación técnica:** Documentación técnica relacionada con el desarrollo del proyecto.
- **Manual de usuario:** Instrucciones para el uso y funcionamiento del sistema.

2. Objetivos del proyecto

Este proyecto tiene como objetivo desarrollar una aplicación móvil capaz de detectar y reportar desperfectos en la vía mediante técnicas de visión artificial. Para ello, se definen dos grupos de objetivos: generales (orientados a las funcionalidades y experiencia de usuario) y técnicos (centrados en la implementación y desarrollo).

2.1. Objetivos generales

- Diseñar y desarrollar una aplicación intuitiva y accesible que permita a los usuarios registrar incidencias relacionadas con el estado de las vías públicas.
- Implementar un sistema de gestión de usuarios que incluya registro, inicio de sesión, modificación de datos personales y eliminación de cuenta.
- Permitir la creación de reportes a partir de imágenes capturadas con el dispositivo móvil, indicando la ubicación exacta del desperfecto y clasificándolo mediante inteligencia artificial.
- Facilitar la consulta, seguimiento y gestión de las incidencias registradas por los usuarios.
- Ofrecer opciones de configuración personalizadas, como selección de idioma, modo visual (oscuro o claro) y gestión de notificaciones.
- Incluir elementos de ayuda al usuario, como secciones informativas, así como funcionalidades que promuevan la difusión de la aplicación.

2.2. Objetivos técnicos

- Desarrollar e integrar un modelo de inteligencia artificial que clasifique correctamente distintos tipos de desperfectos en la vía (grietas, baches, etc.) a partir de imágenes.
- Incorporar tecnologías y servicios en la nube, como Firebase, para la gestión de usuarios, almacenamiento de datos y de imágenes.
- Implementar la funcionalidad de geolocalización para capturar con precisión la ubicación de cada incidencia reportada.
- Garantizar que la aplicación ofrezca una experiencia de usuario fluida y responsiva, con integración eficiente entre la inteligencia artificial, la interfaz y los servicios de backend.
- Facilitar la generación y envío de reportes completos (imagen, clasificación del desperfecto, ubicación) hacia las autoridades competentes.
- Realizar pruebas y validaciones para asegurar la calidad, precisión y rendimiento del modelo de inteligencia artificial y de la aplicación en general.

3. Conceptos teóricos

En este capítulo se presentan los conceptos fundamentales relacionados con la inteligencia artificial (IA) que son necesarios para la comprensión y desarrollo del proyecto. Se abordan los principios básicos del aprendizaje automático y profundo, el funcionamiento de los modelos de detección de objetos, y aspectos relevantes en el entrenamiento de modelos, como el método de *early stopping*.

3.1. Inteligencia Artificial y Aprendizaje Automático

La inteligencia artificial es un área de la informática que busca desarrollar sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el reconocimiento de patrones, la toma de decisiones o la interpretación de imágenes. Dentro de la IA, el aprendizaje automático (machine learning) es la rama que permite a las máquinas aprender a partir de datos, sin ser explícitamente programadas para cada tarea.

3.2. Aprendizaje Profundo (Deep Learning)

El aprendizaje profundo es una subárea del aprendizaje automático que utiliza redes neuronales artificiales con múltiples capas (profundas) para modelar y aprender representaciones complejas de los datos. Según IBM (2024), este enfoque es capaz de procesar grandes cantidades de datos no estructurados y extraer automáticamente características relevantes sin intervención manual [4]. Estas redes son especialmente efectivas en tareas

como el reconocimiento de imágenes, voz y procesamiento de lenguaje natural.

3.3. Detección de Objetos y Box Bounding

La detección de objetos es una tarea central dentro de la visión por ordenador, cuyo objetivo es identificar y localizar objetos específicos dentro de una imagen. Para ello, los modelos generan cuadros delimitadores conocidos como *bounding boxes*, los cuales indican la posición y tamaño de los objetos detectados.

Según MSMK (2023), las *bounding boxes* se representan mediante coordenadas que definen los extremos del rectángulo que rodea al objeto, facilitando así su clasificación y seguimiento [14]. Cada cuadro se acompaña de una etiqueta (clase) y una puntuación de confianza, que expresan la categoría del objeto y la certeza del modelo sobre su predicción.

3.4. Entrenamiento de Modelos de IA

El proceso de entrenamiento consiste en ajustar los parámetros de un modelo para que aprenda a realizar una tarea específica, como clasificar imágenes o detectar objetos. Para ello, se utilizan conjuntos de datos etiquetados que sirven como referencia.

Durante el entrenamiento, el modelo evalúa su desempeño en los datos de entrenamiento y, en ocasiones, en datos de validación, con el fin de evitar el sobreajuste (*overfitting*). Una técnica útil en este contexto es el *early stopping*, que permite detener el entrenamiento cuando la precisión en los datos de validación deja de mejorar, evitando así que el modelo pierda capacidad de generalización.

3.5. Modelo MobileNetV2

MobileNetV2 es una arquitectura de red neuronal profunda diseñada por Google con el objetivo de equilibrar precisión y eficiencia computacional. Está especialmente orientada a dispositivos con recursos limitados, como smartphones o sistemas embebidos.

Arquitectura y funcionamiento

Según Sandler et al. (2018), MobileNetV2 se basa en bloques llamados *inverted residuals* con *linear bottlenecks* [12]. La arquitectura invierte el enfoque tradicional de las redes residuales al expandir los canales primero, aplicar una convolución *depthwise separable*, y luego reducir la dimensión con una convolución 1x1 lineal. Esta estructura permite una reducción significativa en el número de operaciones sin sacrificar precisión.

Aplicación en detección de objetos

MobileNetV2 suele utilizarse como extractor de características en arquitecturas de detección de objetos como SSD (Single Shot Detector). El modelo primero procesa la imagen para identificar patrones relevantes y luego un módulo adicional predice los cuadros delimitadores, clases de objetos y puntuaciones de confianza. Esto permite su uso en aplicaciones de detección en tiempo real.

Ventajas para aplicaciones móviles

Gracias a su diseño optimizado, MobileNetV2 ofrece múltiples beneficios para dispositivos móviles:

- **Tamaño reducido:** permite su integración en aplicaciones móviles con limitaciones de memoria.
- **Alta velocidad de inferencia:** procesamiento de imágenes en tiempo real sin necesidad de hardware especializado.
- **Facilidad de reentrenamiento:** es posible ajustar el modelo a conjuntos de datos específicos para mejorar el rendimiento en casos concretos.

Por estas razones, MobileNetV2 se ha convertido en una solución ideal para proyectos que requieren eficiencia y rendimiento en contextos móviles.

4. Técnicas y herramientas

Esta parte de la memoria tiene como objetivo presentar las técnicas, metodologías y las herramientas de desarrollo que se han utilizado para llevar a cabo el proyecto. En cada una de ellas se explica una comparativa entre distintas opciones disponibles, así como la justificación de las elecciones realizadas.

4.1. Android Studio

Para el desarrollo de la aplicación móvil he elegido **Android Studio** [1], que es el entorno oficial para crear aplicaciones Android. Esta elección se fundamenta en varias razones basadas en mis necesidades y experiencia.

Android Studio es una aplicación de escritorio que funciona en Windows, macOS y Linux, y está basada en IntelliJ IDEA. Cuenta con un editor de código muy completo que soporta Kotlin y Java, lenguajes que ya había estudiado y con los que me siento cómodo. Además, incluye un emulador integrado que me ha permitido probar la aplicación sin necesidad de usar dispositivos físicos, agilizando así el proceso de desarrollo y depuración.

La integración con herramientas como Gradle para la gestión de dependencias y Firebase para servicios en la nube, es directa y muy sencilla, lo que facilita la implementación de funcionalidades complejas sin salir del entorno de desarrollo.

A pesar de que Android Studio es una herramienta que consume muchos recursos y puede resultar pesada para proyectos muy simples, considero que para mi proyecto, que incluye backend, frontend y conexión con servicios externos, es la opción más robusta y eficiente.

En resumen, elegí Android Studio porque:

- Me permite trabajar con lenguajes que ya conozco (Java/Kotlin), reduciendo la curva de aprendizaje.
- Su emulador integrado facilita el desarrollo y las pruebas.
- Cuenta con soporte oficial y una comunidad amplia que me ha servido para resolver dudas y aprender.
- La integración nativa con Firebase y otras herramientas acelera la implementación de funcionalidades.

Por estas razones, considero que Android Studio ha sido la mejor opción para el desarrollo de esta aplicación móvil.

4.2. Servicios en la nube

Para el desarrollo de la aplicación, he utilizado **Firebase** [5] en su versión gratuita. Aunque esta opción tiene algunas limitaciones en cuanto a capacidad y funcionalidades, me ha resultado muy útil para el desarrollo inicial y la fase de prototipo de la aplicación.

Firebase ofrece varias herramientas que facilitan la implementación rápida y escalable de servicios básicos, sin necesidad de montar servidores o infraestructura propia. Entre las funcionalidades que he utilizado destacan:

- **Firebase Authentication:** Me ha permitido implementar un sistema de autenticación sencillo y seguro para gestionar los usuarios de la aplicación sin complicaciones.
- **Firebase Realtime Database:** Una base de datos en tiempo real que sincroniza y almacena los datos de forma eficiente, lo que es clave para mantener la información actualizada al instante.
- **Firebase Storage:** Un servicio para almacenar y gestionar archivos, como imágenes o documentos, de manera rápida y fácil.
- **Firebase Cloud Functions:** Funciones que se ejecutan en la nube cuando se produce un evento determinado, como una modificación en la base de datos. En este proyecto, las he utilizado para enviar notificaciones push automáticas a los usuarios cuando una incidencia cambia de estado o es respondida por un administrador.

Además, el sistema se ha dividido en dos proyectos diferenciados:

- Una **aplicación móvil** destinada al ciudadano, desarrollada en Android con Java, que permite reportar incidencias, hacer seguimiento y visualizar su evolución.
- Una **página web para el administrador**, desde la cual los gestores del sistema pueden revisar, clasificar y resolver incidencias. Esta interfaz también está conectada a Firebase, permitiendo la gestión de los datos en tiempo real.

Si bien la versión gratuita tiene ciertas restricciones —como límites en el almacenamiento o en el número de peticiones por segundo— estas no han supuesto un problema para el alcance de este proyecto. Gracias a Firebase, he podido centrarme en la lógica de la aplicación y su funcionalidad, sin tener que desarrollar ni mantener una infraestructura de backend desde cero.

En resumen, Firebase ha sido una herramienta muy práctica y eficiente para esta etapa del proyecto, permitiendo un desarrollo ágil tanto en la parte móvil como en la plataforma web.

4.3. Framework de Inteligencia Artificial

Para integrar inteligencia artificial en mi aplicación móvil, he optado por utilizar **TensorFlow Lite** [2], una versión ligera del popular framework TensorFlow, desarrollada por Google y pensada específicamente para dispositivos móviles y embebidos.

Motivo de la elección

Después de analizar varias opciones disponibles, decidí usar TensorFlow Lite porque se adapta perfectamente a las necesidades de mi proyecto. Mi aplicación necesita ejecutar un modelo de detección de objetos directamente en el dispositivo móvil, sin depender de servidores externos o de una conexión a Internet constante. TensorFlow Lite me permite hacer esto de forma eficiente, rápida y con un bajo consumo de recursos.

Además, gracias a su diseño optimizado, las inferencias del modelo se realizan en tiempo real, lo que mejora la experiencia del usuario al obtener resultados inmediatos tras capturar una imagen. Esta capacidad de procesamiento local también ayuda a preservar la privacidad, ya que no es necesario enviar las imágenes a un servidor externo para ser analizadas.

Ventajas principales que me han llevado a usarlo

- **Eficiencia en móviles:** TensorFlow Lite está optimizado para ejecutarse en dispositivos con recursos limitados, como móviles Android, lo cual encaja con el entorno de ejecución de mi aplicación.
- **Inferencia en tiempo real:** El modelo entrenado se ejecuta directamente en el móvil, lo que permite detectar incidencias de forma inmediata tras tomar una foto.
- **Privacidad del usuario:** Al no requerir conexión a la nube para procesar las imágenes, se evita subir contenido sensible, lo que refuerza la seguridad y privacidad de los usuarios.
- **Compatibilidad y documentación:** TensorFlow Lite se integra fácilmente con Android Studio, tiene buena documentación oficial y una comunidad activa, lo que ha facilitado su implementación en el proyecto.
- **Conversión sencilla del modelo:** He podido convertir mi modelo de entrenamiento al formato `.tflite` usando las herramientas proporcionadas por TensorFlow sin complicaciones.

Aplicación en el proyecto

En mi aplicación, el modelo de IA entrenado con imágenes de desperfectos en la vía pública se ha convertido al formato compatible con TensorFlow Lite y se ha integrado en el proyecto Android. Gracias a esto, cada vez que el usuario toma una foto, el sistema es capaz de identificar automáticamente si hay una incidencia y cuál es su tipo. Todo este proceso ocurre localmente en el dispositivo, sin depender de servidores externos, lo cual mejora la velocidad de respuesta y la autonomía de la app.

En resumen, he elegido TensorFlow Lite porque es una solución ligera, rápida y adecuada para ejecutar modelos de inteligencia artificial en tiempo real dentro de una aplicación móvil, lo que ha sido clave para cumplir con los objetivos de mi TFG.

4.4. Modelo de IA usado

Tras seleccionar TensorFlow como framework para el desarrollo del modelo de inteligencia artificial, el siguiente paso fue elegir un modelo pre-entrenado adecuado para la clasificación de imágenes en la aplicación móvil.

Para ello, evalué varios modelos ampliamente reconocidos y compatibles con TensorFlow Lite, entre ellos MobileNetV2 y EfficientNet V2.

Al final escogí **MobileNetV2**, principalmente por el balance óptimo que ofrece entre precisión, tamaño del modelo y velocidad de inferencia, aspectos cruciales para aplicaciones móviles que deben funcionar con recursos limitados y en tiempo real.

MobileNetV2 es un modelo ligero y eficiente, diseñado específicamente para dispositivos con capacidad computacional limitada, como smartphones y tablets. Además, puede ser fácilmente reentrenado con datasets personalizados, como los utilizados en este proyecto, lo que permite adaptar el modelo a las características específicas de las incidencias urbanas que se quieren detectar.

Una ventaja importante es la extensa documentación, soporte y comunidad que rodea a MobileNetV2, así como su compatibilidad nativa con TensorFlow Lite. Esto facilita la integración directa en entornos de desarrollo Android Studio y permite optimizar el modelo para un desempeño eficiente en dispositivos móviles.

Las pruebas comparativas realizadas, cuyos resultados se detallan en el anexo Manual de programador en la sección de Pruebas del sistema, mostraron que aunque EfficientNet V2 puede alcanzar una precisión ligeramente superior, MobileNetV2 destaca por su velocidad de inferencia significativamente mayor y un tamaño más reducido. Estas características hacen que MobileNetV2 sea más adecuado para garantizar una respuesta rápida y eficiente en tiempo real, sin sacrificar demasiado la precisión, lo cual es fundamental para la experiencia de usuario en la aplicación.

En resumen, MobileNetV2 fue seleccionado como modelo base debido a:

- Su excelente equilibrio entre precisión, tamaño y velocidad, ideal para dispositivos móviles.
- La facilidad para reentrenarlo con datasets específicos del proyecto, mejorando su desempeño en tareas concretas.
- Amplia documentación y soporte para su uso con TensorFlow Lite y Android Studio.
- Su capacidad para ejecutarse de forma rápida y eficiente, permitiendo un procesamiento en tiempo real dentro de la aplicación.

- Resultados experimentales que respaldan su mejor adecuación frente a otros modelos evaluados, como EfficientNet V2.

Esta combinación de ventajas garantiza un modelo robusto y eficiente para la detección y clasificación de incidencias urbanas en la vía pública.

4.5. Dataset utilizados

Para el desarrollo del modelo de inteligencia artificial de la aplicación, he utilizado tres dataset distintos, cada uno enfocado en aspectos específicos del problema a resolver: clasificación de incidencias, detección de humanos y filtrado de objetos no relacionados con incidencias.

- **Dataset para clasificación de incidencias:** Este conjunto de datos se empleó para entrenar el modelo encargado de clasificar los tipos de desperfectos o incidencias detectadas en las imágenes. Está respaldado por un estudio científico que valida su calidad y utilidad para este tipo de tareas [15]. Este dataset incluye anotaciones en formato XML para facilitar el entrenamiento y la validación.
- **Dataset para detección de humanos:** Para garantizar la privacidad y evitar almacenar imágenes que contengan personas, se usó un dataset específico de detección de rostros humanos. Esto permite filtrar o procesar adecuadamente las imágenes con presencia humana. Este dataset, disponible en Kaggle [6], utiliza anotaciones en formato TXT para validar los *bounding boxes* de los rostros.
- **Dataset para detección de objetos no relacionados con incidencias:** Para evitar falsos positivos y filtrar objetos que no corresponden a una incidencia, se empleó un dataset que contiene una amplia variedad de objetos comunes. Esto ayuda a mejorar la precisión del modelo y reducir errores. Este dataset, también disponible en Kaggle [11], utiliza anotaciones en formato XML.

La combinación de estos tres datasets ha permitido construir un modelo robusto y efectivo para la detección y clasificación de incidencias urbanas, respetando la privacidad y mejorando la calidad de las predicciones.

4.6. Control de versiones

Para gestionar el desarrollo del proyecto, he utilizado un repositorio en **GitHub** como plataforma principal de control de versiones. El repositorio está disponible en **GitHub** [17]. Esta herramienta me ha permitido mantener un historial de los cambios realizados a lo largo del proyecto, facilitando la detección de errores y posibilitando la colaboración.

Gestión de tareas y documentación

En GitHub he utilizado diversas funcionalidades para el seguimiento y la organización del trabajo:

- **Issues:** Para registrar, priorizar y dar seguimiento a tareas, errores y nuevas funcionalidades.
- **README:** Documentación centralizada y actualizada que facilita la comprensión y el uso del código.
- **Branches y Pull Requests:** Para desarrollar nuevas funcionalidades y corregir errores de forma aislada, asegurando revisiones antes de integrar cambios al código principal.

Calidad del código

Para garantizar la calidad del código fuente, se ha integrado **SonarQube** en el propio Android Studio. Esta herramienta me ha permitido monitorizar la calidad del proyecto y analizar estáticamente el código para identificar:

- Errores y vulnerabilidades de seguridad.
- Problemas de mantenibilidad y complejidad.
- Código duplicado y malas prácticas.

Gracias a esta integración, se ha asegurado una calidad alta en el código del proyecto, lo que facilita el mantenimiento y la evolución futura del mismo.

5. Aspectos relevantes del desarrollo del proyecto

Este capítulo expone en profundidad las decisiones técnicas más importantes tomadas durante el desarrollo del proyecto, destacando especialmente el uso de inteligencia artificial para la clasificación de desperfectos urbanos y el desarrollo de una plataforma web de administración para la gestión de incidencias.

Se detallan los aspectos del ciclo de vida seguido, así como el diseño e implementación tanto del modelo de IA como de la infraestructura web que da soporte al usuario administrador del sistema.

5.1. Desarrollo e integración del modelo de clasificación con MobileNetV2

Uno de los elementos clave del proyecto fue la incorporación de un modelo de inteligencia artificial capaz de clasificar imágenes de desperfectos en la vía pública desde un dispositivo móvil. Para ello, opté por el uso de un modelo preentrenado basado en MobileNetV2, al que se le aplicó un proceso de reentrenamiento personalizado.

Selección y entrenamiento del modelo

Tras comparar distintas alternativas disponibles elegí usar **MobileNetV2** por su excelente compromiso entre precisión, tamaño y velocidad, aspectos fundamentales en una aplicación móvil.

El modelo fue reentrenado con un conjunto de datos personalizado que incluye imágenes de doce clases de incidencias urbanas:

1. 0 — Humano
2. D00 — Grieta longitudinal en zona de rodadura
3. D40 — Deformaciones y desprendimientos
4. 1 — Sin incidencia
5. D50 — Alcantarillas
6. D20 — Grieta piel de cocodrilo
7. D10 — Grieta transversal de intervalo regular
8. D44 — Señalización horizontal deteriorada
9. Repair — Zona reparada
10. D43 — Señalización horizontal deteriorada
11. D01 — Grieta longitudinal en junta de construcción
12. D11 — Grieta transversal en junta de construcción

Resumen del proceso de entrenamiento:

- Total de imágenes utilizadas: 84.692
- División del conjunto de datos:
 - Entrenamiento: 67.753 imágenes (80 %)
 - Validación: 16.939 imágenes (20 %)
- Duración del entrenamiento: aproximadamente 4 horas
- Técnica: **transfer learning**, congelando las capas convolucionales base y entrenando únicamente las capas superiores
- Técnicas adicionales: **early stopping** (paciencia = 5)

Evolución del entrenamiento

Durante el entrenamiento se observó una evolución estable tanto en la precisión como en la pérdida. El modelo convergió en la época 8 sin mostrar síntomas de sobreajuste, gracias al uso del conjunto de validación y la técnica de `early stopping`.

Evaluación del modelo

Una vez entrenado, se evaluó el modelo sobre el conjunto de validación. Los resultados obtenidos fueron:

- **Precisión media (accuracy):** 63,75 %
- **Recall promedio:** 56,77 %
- **F1-score promedio:** 53,00 %

Se generó el siguiente `classification report`:

[language=Python, caption=Classification report del modelo]

	precision	recall	f1-score	support
0	0.8564	0.8953	0.8754	2970
D00	0.6103	0.5929	0.6015	3009
D40	0.5813	0.5678	0.5745	1291
1	0.8810	0.8570	0.8688	2999
D50	0.3185	0.4348	0.3677	706
D20	0.4549	0.4181	0.4357	2160
D10	0.4963	0.4540	0.4742	2392
D44	0.4649	0.4691	0.4670	1002
Repair	0.5806	0.7826	0.6667	207
D43	0.3984	0.6135	0.4831	163
D01	0.4364	0.7273	0.5455	33
D11	0.0000	0.0000	0.0000	7
accuracy			0.6375	16939
macro avg	0.5066	0.5677	0.5300	16939
weighted avg	0.6395	0.6375	0.6373	16939

También se generó la siguiente matriz de confusión:

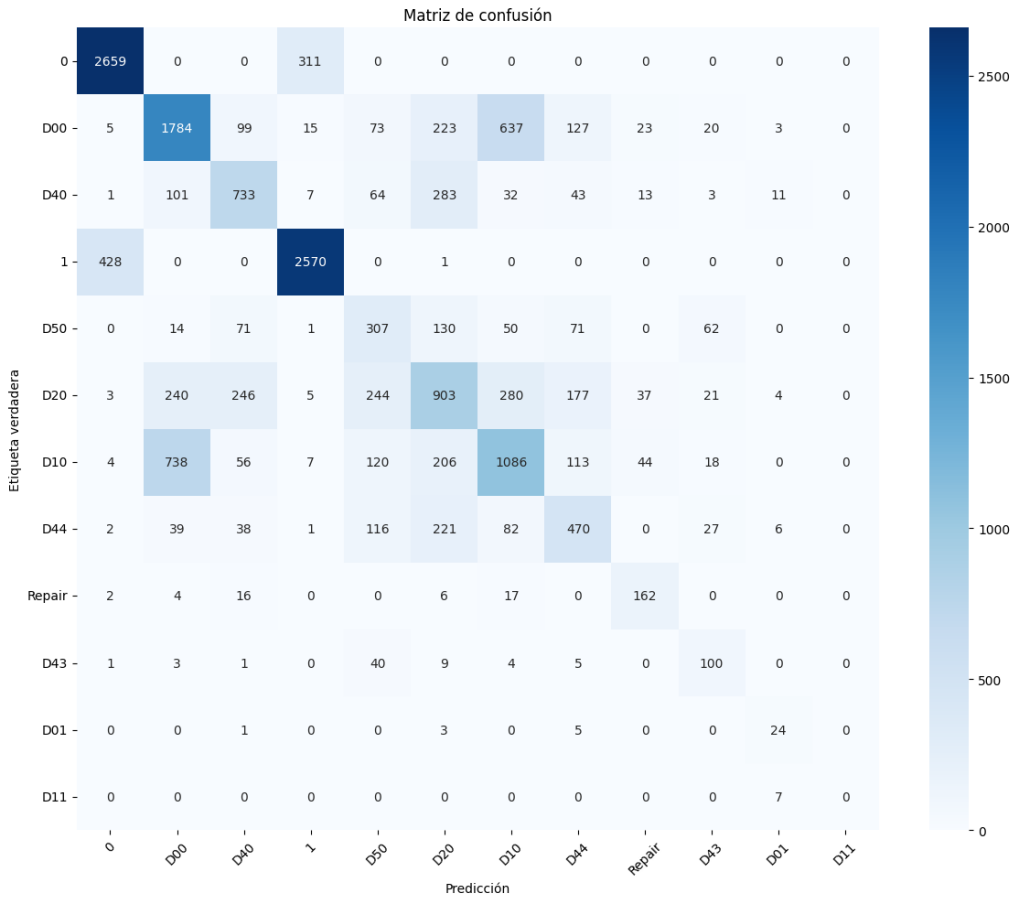


Figura 5.1: Matriz de confusión del modelo

Como puede observarse, el modelo obtiene mejores resultados en clases con mayor número de ejemplos y en aquellas con características más distintivas, como la clase "humano."o "sin incidencia", frente a otras más similares entre sí como los distintos tipos de grietas.

Conversión e integración en Android

Tras validar el modelo, este fue exportado al formato TensorFlow Lite (.tflite) utilizando la herramienta TFLiteConverter. Luego, fue integrado en el proyecto Android mediante la biblioteca TensorFlow Lite Task Library, permitiendo realizar inferencias en tiempo real desde la cámara o la galería del dispositivo.

5.2. Creación de la plataforma web para el usuario master

Además de la aplicación móvil, se desarrolló una plataforma web destinada a los usuarios con rol de administrador (denominados *usuarios master*), quienes tienen la responsabilidad de visualizar, gestionar y dar seguimiento a las incidencias reportadas por los usuarios desde la aplicación móvil.^[16]

Objetivo de la plataforma

El propósito principal de esta interfaz es facilitar a los gestores de mantenimiento urbano una visión clara y ordenada del estado de las calles, reportado por los ciudadanos. Desde la web pueden:

- Visualizar todas las incidencias registradas, con su imagen, tipo, fecha y ubicación
- Cambiar el estado de una incidencia: pendiente, en revisión, resuelta, etc.
- Ordenar las incidencias por ubicación, fecha o estado

Tecnologías utilizadas

- **Frontend:** HTML
- **Backend:** Firebase Functions, aunque la mayor parte de la lógica está en el cliente gracias al uso de Firebase como BaaS
- **Base de datos:** Firestore , integrada directamente con el frontend
- **Autenticación:** Firebase Authentication con sistema de control de roles para distinguir usuarios normales y administradores

Diseño e interfaz

Se desarrolló una interfaz limpia y funcional, optimizada para escritorio, con una estructura basada en tarjetas que muestra cada incidencia. Estas tarjetas contienen:

- ID de la incidencia

- Correo electrónico del usuario
- Calle de la incidencia
- Coordenadas de la ubicación
- Tipo de incidencia
- Fecha del reporte
- Fotografía de la incidencia
- Selector para actualizar el estado

Panel de Incidencias

ID	Email	Calle	Coordenadas	Tipo	Fecha	Foto	Estado
3x1dqRmrstlbS...	marcosgomezve...	Calle Guiomar F...	Lat: 42.3600026...	Grieta longitudin...	2025-07-03 21:5...		Pendiente ▾
A56ARUxMFct2...	marcosgomezve...	Calle Picote, Ar...	Lat: 41.6834816...	Sin incidencia	2025-06-25 21:1...		Pendiente ▾
B8uwbyVFCWP...	marcosgomezve...	Calle Victoria Ba...	Lat: 42.3599738...	Grieta	2025-07-03 22:1...		Pendiente ▾
CwejfHA6fL93V...	marcosgomezve...	Calle Guiomar F...	Lat: 42.3599953...	Grieta	2025-06-25 10:4...		Pendiente ▾
EFLGxevlVmR3...	marcosgomezve...	E.G.Q. Colegio, ...	Lat: 42.3529067...	Grieta	2025-06-25 14:3...		Pendiente ▾

Figura 5.2: Web Administrador

Interacción con Firebase

Todas las operaciones de lectura, filtrado y actualización de estado se realizan en tiempo real gracias a la integración con Firestore. Esto permite que, si un usuario cambia el estado de una incidencia desde la plataforma web, el cambio se refleje automáticamente en la aplicación móvil cuando el usuario consulta su historial.

El sistema fue diseñado de manera que solo los usuarios con permisos puedan realizar modificaciones, empleando reglas de seguridad en Firestore que validan el rol de cada usuario.

Gestión de estados

Los estados definidos para las incidencias son los siguientes:

- **Pendiente:** estado inicial tras el reporte
- **En proceso:** el administrador ha tomado nota del problema
- **Resuelto:** el desperfecto ha sido reparado

6. Trabajos relacionados

En este capítulo se analizan aplicaciones móviles existentes orientadas al reporte de problemas en infraestructuras públicas, como baches, iluminación deficiente o desperfectos en carriles bici. El objetivo es identificar características comunes, fortalezas, limitaciones y oportunidades de mejora en el diseño y funcionalidad de estas soluciones.

6.1. Aplicaciones Analizadas

Se revisaron cinco aplicaciones representativas mediante un análisis funcional y de experiencia de usuario (UX):

Municipio Inteligente [3]

- **Plataforma:** Web
- **Características principales:**
 - Reporte de incidencias urbanas mediante formulario
 - Posibilidad de adjuntar fotos
 - Visualización en mapa
 - Gestión y seguimiento por parte del ayuntamiento
- **Ventajas:**
 - Sistema orientado a municipios españoles
 - Integración directa con servicios municipales

- Interfaz simple y accesible desde navegador
- **Desventajas:**
 - No tiene aplicación móvil nativa
 - Requiere acceso manual para subir fotos desde dispositivos móviles
 - Sin funcionalidades avanzadas como IA o geolocalización automática

Waze [7]

- **Plataforma:** Android, iOS
- **Características principales:**
 - App para viajar
 - Reporte de incidencias en tiempo real
 - Visualización de incidencias reportadas por otros usuarios
 - Visualización en el mapa con ubicación precisa
 - Categorización de la incidencia
- **Ventajas:**
 - Gran comunidad activa de usuarios que reportan incidencias constantemente
 - Actualizaciones en tiempo real de la situación del tráfico
 - Interfaz de usuario intuitiva y fácil de usar
 - Funciones de navegación integradas con el reporte de incidencias
- **Desventajas:**
 - Dependencia de la comunidad para mantener la precisión de la información
 - No tiene un sistema de validación oficial para las incidencias
 - La calidad de los reportes puede ser inconsistente debido a la naturaleza abierta de la app
 - No se puede subir fotos de las incidencias
 - No usa IA para detectar las incidencias

TizAPP [13]

- **Plataforma:** Android, iOS
- **Características principales:**
 - Reporte de incidencias urbanas con fotos
 - Mapa interactivo con geolocalización precisa
 - Sistema de categorización de incidencias
 - Notificación y seguimiento del estado de la incidencia
- **Ventajas:**
 - Interfaz amigable y fácil de usar para los ciudadanos
 - Comunicación directa con las autoridades locales
 - Soporte para que los usuarios puedan seguir el progreso de sus reportes
- **Desventajas:**
 - Limitada en algunos países en cuanto a cobertura de servicios y autoridades locales
 - No tiene IA para el análisis automático de fotos

FixMyStreet [9]

- **Plataforma:** Android, iOS, Web
- **Características principales:**
 - Subida de fotos del problema
 - Detección de ubicación mediante GPS
 - Categorización de la incidencia
 - Visibilidad pública de los reportes
- **Ventajas:**
 - Interfaz limpia e intuitiva
 - Seguimiento en tiempo real
 - Código abierto, su GitHub [10]

- **Desventajas:**

- Soporte de ubicación limitado en algunas regiones
- Sin notificaciones push para actualizaciones
- El usuario tiene que especificar el tipo de incidencia

SeeClickFix[8]

- **Plataforma:** Android, iOS, Web

- **Características principales:**

- Reporte con fotos
- Mapas interactivos
- Comunicación con autoridades locales

- **Ventajas:**

- Fuerte integración con gobiernos locales
- Funciones de participación comunitaria

- **Desventajas:**

- Requiere cuenta para reportar
- Poco uso fuera de EE.UU.
- No usa IA para detectar la incidencia

6.2. Patrones UX Identificados

Las aplicaciones estudiadas presentan patrones comunes que pueden servir de base para el diseño del sistema propuesto:

- **Flujo de reporte centrado en la foto:** Las aplicaciones analizadas se centran en permitir que los usuarios suban fotos de manera rápida y fácil para ilustrar el problema, lo cual facilita una identificación más clara del reporte. Esta es una característica fundamental en la mayoría de las apps analizadas.
- **Ubicación mediante mapa:** Utilizan mapas interactivos con pines que los usuarios pueden colocar fácilmente para definir la ubicación del problema, lo que mejora la precisión de los reportes.

- **Categorías predefinidas:** Muchas aplicaciones permiten seleccionar una categoría específica para el tipo de problema, lo que reduce el tiempo de ingreso de datos y mejora la organización.
- **Indicadores visuales del estado:** Las aplicaciones utilizan indicadores visuales para mostrar el estado de cada incidencia, lo que proporciona transparencia y mantiene al usuario informado sobre el progreso de su reporte.

6.3. Comparativa

La siguiente tabla resume las principales características de varias aplicaciones orientadas a reportar problemas en infraestructuras públicas. Se comparan aspectos clave como el uso de fotos, geolocalización, inteligencia artificial (IA), y la plataforma de uso. Esta comparativa permite identificar fortalezas, debilidades y oportunidades de mejora en el ecosistema actual.

App	Plataforma	Fotos	Geo	IA
MunicipioInteligente	Web	Sí	No	No
Waze	Android/iOS	No	Sí	No
TizAPP	Android/iOS	Sí	Sí	No
FixMyStreet	Android/iOS/Web	Sí	Sí	No
SeeClickFix	Android/iOS/Web	Sí	Sí	No

Tabla 6.1: Comparativa de aplicaciones para reporte de incidencias urbanas

6.4. Vacíos y Oportunidades

Durante el análisis se detectaron áreas que podrían mejorarse:

- **Falta de IA:** Integrar IA permitiría clasificar automáticamente las incidencias a partir de fotos, evitando que el usuario seleccione el tipo de problema. Esto agiliza el proceso, mejora la precisión y abre la posibilidad de admitir videos para análisis en tiempo real.
- **Interacción limitada:** Muchas apps no permiten una comunicación fluida entre los usuarios y las autoridades locales. Un sistema de chat en tiempo real o una función de comentarios podría mejorar la comunicación y el seguimiento de los reportes.

- **Baja personalización:** La mayoría de las apps utilizan categorías predefinidas limitadas para clasificar los problemas reportados. Se podría ofrecer una mayor personalización, permitiendo a los usuarios crear nuevas categorías o subcategorías según el tipo de incidencia.

6.5. Conclusión

El análisis de aplicaciones actuales evidencia una carencia de soluciones que combinen accesibilidad, uso de IA y una experiencia de usuario rica, lo que justifica el desarrollo del sistema propuesto en este TFG.

7. Conclusiones e informe crítico

7.1. Conclusiones

A lo largo del desarrollo de este trabajo de fin de grado he podido tomar conciencia del esfuerzo que conlleva crear una aplicación móvil, incluso cuando esta tiene un enfoque aparentemente sencillo. Detrás de cada funcionalidad visible al usuario existe una gran cantidad de código, planificación y pruebas. Muchos de estos aspectos pasan desapercibidos, pero son fundamentales para lograr una aplicación funcional, estable y segura.

He aprendido que incluso en aplicaciones de tamaño medio o pequeño, es completamente normal que puedan surgir errores o fallos durante el desarrollo o tras su despliegue. La complejidad del software moderno, especialmente cuando incluye integración con servicios externos, inteligencia artificial y sistemas de geolocalización, hace que sea difícil cubrir todos los casos posibles mediante pruebas manuales o automáticas. Esta experiencia me ha permitido valorar más profundamente el trabajo que hay detrás de cada aplicación que utilizamos en nuestro día a día.

7.2. Informe crítico y posibles mejoras

Durante el desarrollo y análisis del proyecto han surgido algunas ideas de mejora o aspectos que podrían haberse abordado de forma más completa si se dispusiera de más tiempo o recursos:

- **Privacidad mediante detección avanzada:** Actualmente, la IA detecta el tipo de incidencia, pero una mejora importante sería implementar una detección de bounding boxes (por ejemplo, con modelos

entrenados mediante Roboflow) para identificar si aparece una persona en la imagen. En ese caso, se podría evitar subir dicha imagen a la nube (Firebase Storage) para preservar la privacidad de las personas.

- **Automatización y testeo:** La creación de tests automáticos es limitada, ya que actualmente requiere ejecutar un emulador para poder probar la aplicación. Una futura mejora sería implementar un sistema de pruebas más robusto que permita la validación de funcionalidades clave sin necesidad de entornos pesados o manuales.
- **Ampliación del dataset:** Otra posible línea de mejora sería incorporar a la página web del sistema una herramienta para reclasificar o validar nuevas imágenes mediante Roboflow o CVAT. De este modo, se podría generar un dataset más amplio y diverso que permitiría reentrenar el modelo IA con mayor precisión y robustez.
- **Gestión de errores:** Aunque se ha manejado el control de errores básicos, se podría seguir mejorando la tolerancia a fallos, especialmente en la comunicación con servicios externos como Firebase o la API de IA, implementando reintentos o notificaciones más específicas al usuario.

En resumen, este proyecto me ha permitido aprender no solo a nivel técnico, sino también a nivel organizativo, valorando la planificación, la modularidad y la importancia de escribir código mantenible y escalable. Las mejoras aquí propuestas servirían como una base sólida para futuras versiones del sistema.

Bibliografía

- [1] Android Studio Authors. Android studio, 2025.
- [2] TensorFlow Authors. Tensorflow lite, 2025.
- [3] AYESA. Municipio inteligente, 2023.
- [4] IBM Corporation. Explciación deep learning, 2024.
- [5] Google. Firebase, 2025.
- [6] Ashwin Gupta. Human faces dataset, 2023.
- [7] Google Inc. Waze - navegación en tiempo real, 2024.
- [8] SeeClickFix Inc. Seeclickfix - reporte ciudadano de incidencias, 2024.
- [9] mySociety. Fixmystreet, 2024.
- [10] mySociety. Fixmystreet - repositorio en github, 2024.
- [11] Ngan. Voc devkit 2007 and 2012 dataset, 2012.
- [12] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. *arXiv preprint arXiv:1801.04381*, 2018.
- [13] TizAPP. Tizapp - aplicación de gestión de incidencias urbanas, 2023.
- [14] MSMK University. Explciación bounding box, 2023.
- [15] Author Unknown. A validated dataset for classification of urban incidences. *Royal Meteorological Society Geoscience Data Journal*, 2022.

- [16] Marcos Gómez Vega. Pagina web usuario administrador, 2025.
- [17] Marcos Gómez Vega. Repositorio del proyecto en github, 2025.